Tarea 2: Torneo de Komikan Concurrente

Sistemas Operativos | Universidad Diego Portales
Integrantes: Benjamín Zúñiga, Tomas Nuñez | Fecha: Octubre 2026

1. Diseño del Esquema de Sincronización

Compilación Estricta y Configuración (`src/config.c`)

Compilamos el proyecto de forma estricta en C17 (`-std=c17`), activando todas las advertencias del compilador (`-Wall -Wextra`) y soporte de hilos POSIX (`-pthread`). Para manejar la configuración, implementamos el parser en `src/config.c` (`src/config.h`). Este módulo lee el archivo pasado por parámetro, valida los rangos de cada opción (como delays y posición del puma) y asigna valores por defecto seguros si el archivo falta o contiene errores. Para evitar advertencias de funciones POSIX con ISO C estricto, agregamos `#define _DEFAULT_SOURCE` al inicio de los archivos.

Arquitectura y Jerarquía de Hilos

Organizamos el servidor en dos niveles de hilos (`src/torneo.c`, `src/mesa.c`). En el nivel superior trabaja el hilo principal (que maneja las señales POSIX), el hilo emparejador (*matchmaker*) que recibe a los N jugadores, y un hilo coordinador por cada una de las M mesas. A nivel de partida, cada mesa crea 13 hilos de fichas (1 puma y 12 perros); el juez es el hilo de la mesa (`src/mesa.c`, `src/paso4.c`). Representamos el tablero como un grafo estático de 28 nodos (25 casillas de la grilla 5×5 y 3 nodos del trono: 25, 26 y 27) en `src/tablero.c`, inicializando la geometría una sola vez mediante `pthread_once`.

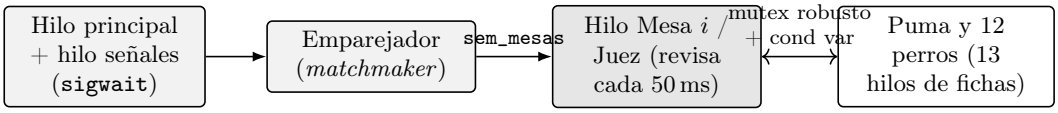


Figura 1: Organización de hilos: del servidor a las mesas (donde el hilo de la mesa actúa como juez) y los 13 hilos de fichas.

Para sincronizar los hilos, evitar espera activa y limpiar recursos, usamos cinco primitivas POSIX:

- **Mutex por mesa y consistencia del tablero (`src/paso3.c`, `src/mesa.c`):** Protegemos la consulta, validación y movimiento en el tablero dentro de una misma sección crítica. Esto evita condiciones de carrera TOCTOU (como en `src/paso2.c`, donde dos hilos leían la misma casilla libre al mismo tiempo y se sobrescribían).
- **Variable de condición y espera pasiva del juez (`src/paso4.c`, `src/mesa.c`):** El juez duerme con `pthread_cond_timedwait` (periodo de 50 ms) para no gastar CPU. Se despierta al recibir avisos de movimiento o cada 50 ms para revisar victorias, latidos y cierres por `SIGINT`.
- **Mutex robusto ante fallos (`src/paso8.c`, `src/mesa.c`):** Usamos `PTHREAD_MUTEX_ROBUST`. Si una ficha muere con el lock tomado, la siguiente recibe `EOWNERDEAD`, llama a `pthread_mutex_consistent` y termina la partida de forma segura (`-1`).
- **Semáforo contador `sem_mesas` y límite de mesas (`src/torneo.c`):** Limitamos las M mesas concurrentes con `sem_timedwait`. Cuando no hay cupos, el emparejador espera dormido en el semáforo sin consumir CPU.
- **Captura de `SIGINT` con `sigwait` y limpieza (`src/torneo.c`):** Bloqueamos `SIGINT` en todos los hilos y creamos un hilo dedicado que la espera con `sigwait`. Al presionar Ctrl+C, avisamos a las mesas, hacemos `join` de los hilos y cerramos dejando `activas=0`.

Emparejamiento y Gestión de Desconexiones (`src/paso5.c`, `src/mesa.c`)

- **Cola de Emparejamiento (`src/paso5.c`):** El emparejador recibe a los N jugadores en una cola sincronizada con `mutex_estado` y `cond_cola`. Arma parejas de a dos (Puma y Perros), pide cupo en `sem_mesas` y si la cantidad de jugadores es impar (verificado manualmente), avisa al jugador sobrante y lo libera sin bloquear nada.
- **Supervisión de Latencia y Heartbeats (`src/mesa.c`):** Cada ficha actualiza su tiempo de latido (*heartbeat*) al mover. El juez en `src/mesa.c` revisa estas marcas cada 50 ms para medir el tiempo de inactividad de cada participante.

- **Manejo de Desconexiones Abruptas (`src/mesa.c`):** Si una ficha supera el tiempo de `TIMEOUT_MS` sin responder (verificado manualmente), el juez en `src/mesa.c` declara abandono, da por terminada la partida y devuelve el cupo de la mesa con `sem_post(&sem_mesas)`.

2. Análisis de Riesgos de Concurrencia

- **Deadlock (Interbloqueo):** Evitamos deadlocks al no pedir locks individuales por casilla, sino un único mutex por mesa. Además, fijamos el orden estricto `mesa` → `log` para imprimir logs sin esperas circulares. Las pruebas con ThreadSanitizer y AddressSanitizer (verificado manualmente) confirmaron 0 carreras de datos en partida normal; 1 en la ruta SIGINT (`torneo.c:75`) y 0 fugas de memoria.
- **Locks Huérfanos:** Comprobamos en `src/paso8.c` que un mutex estándar deja la mesa bloqueada para siempre si un hilo muere dentro de la sección crítica (`paso8 normal`). Con el mutex robusto, atrapamos `EOWNERDEAD`, reparamos el estado y terminamos la partida limpiamente (`paso8 robusto`).
- **Livelock:** Evitamos que los perros y el puma se muevan indefinidamente fijando topes de 30 s y 5000 movimientos por mesa. Las partidas largas (verificado manualmente) cortan correctamente por tope de tiempo.
- **Starvation (Inanición):** Mantenemos las secciones críticas muy cortas (solo lecturas y escrituras del grafo en memoria) y hacemos los retardos de simulación (`usleep`) fuera del lock.

3. Evaluación

- **Complejidad vs. Concurrencia:** Proteger cada uno de los 28 nodos con un lock propio obligaría a tomar hasta 3 locks en orden estricto en cada captura. Preferimos un mutex por mesa porque simplifica la atomicidad y permite paralelismo real a nivel de servidor entre las M mesas concurrentes.
- **Rendimiento y Elección de Mutex Robusto sobre Semáforo:** Al comparar `paso3` y `paso3b`, vimos que el semáforo binario y el mutex rinden igual (3786 vs 3773 movimientos). Sin embargo, elegimos el mutex robusto porque el semáforo no registra qué hilo lo tiene tomado y no permite recuperarse si un hilo muere.

Primitiva	**Qué protege**	**Qué pasa si el dueño muere con el lock tomado**
Mutex común (`paso3`)	Tablero (3786 movs, 0 pérdidas)	Sin `EOWNERDEAD`; mesa colgada (verificado en `paso8`, > 2 s).
Semáforo binario (`paso3b`)	Tablero (3773 movs, 0 pérdidas)	Sin concepto de dueño; la mesa queda trabada para siempre.
Mutex robusto (`paso8`, `mesa`)	Tablero y estado de partida	Retorna `EOWNERDEAD`, recupera lock y aborta con `-1`.
`cond_timedwait` (50 ms)	Juez, victoria y *heartbeats*	Espera pasiva; detecta abandono (`timeout` 300/1700 ms).
Semáforo `sem_mesas`	Cupos de M mesas (`timedwait`)	La mesa abortada libera el cupo; servidor cierra con `activas=0`.

Tabla 1: Comparación de primitivas de sincronización y respuesta ante fallos.

Evaluación del Parser de Configuración y Cobertura de Pruebas

El parser de `src/config.c` lee el archivo pasado por parámetro ignorando líneas vacías y corrigiendo valores fuera de rango. Ejecutamos los casos de prueba (verificado manualmente), cubriendo emparejamientos impares, partidas concurrentes, interrupción con `SIGINT`, desconexión por timeout y fallos de hilos. La validación con TSan y ASan confirmó 0 condiciones de carrera en partida normal; 1 en la ruta SIGINT (`torneo.c:75`), 0 fugas de memoria y un cierre de servidor verificado con `activas=0`.